Level-1 models at a glance

ccd_opt_ml · module PA03, board 101 · October 2026

Task. From an amplifier's short readout trace (and the settings) predict what a WADERs fit of the image will give: read noise **σe**, dark charge **λ** and **gain**. One packet = 4 amplifiers = 4 examples. No model is told which amplifier it sees (except the reference). **Testing:** 5-fold cross-validation *grouped by packet* (a packet's 4 amplifiers are always held out together); scores: error (typical % off), error ÷ that of guessing the average, rank within an amplifier, calibrated coverage of the error bars. Error column: synthetic packets (toy CCD world), 41 packets, mean of 3 datasets; it compares methods, not the real CCD.

Model *(looks at)*	What it does	σe / λ / gain error [%]
Baselines		
mean *(nothing)*	Always predicts the training average. Any useful model must beat it.	57.5/29.2/31.3
settings *(settings only)*	Gaussian process on the settings, no trace. A trace model that cannot beat it has not shown the traces help.	57.5/21.7/31.3
Standard regressors (settings + 85 trace numbers of the amplifier's own and reference traces)		
ridge *(85 numbers)*	Bayesian linear fit with an error bar; simple and hard to fool.	9.5/12.4/0.9
trees *(85 numbers)*	Gradient-boosted decision trees: thresholds and kinks.	9.6/21.0/1.3
gp *(85 numbers)*	Gaussian process: smooth prediction from similar examples, with an honest error bar.	7.9/7.4/0.9
nn *(85 numbers)*	Small ensemble of neural networks; their disagreement is the error bar.	9.7/16.9/1.3
Physics-based (the main models)		
trace_physics *(Allan curve)*	No learning: fits $\sigma(N)^2 = a^2/N + b^2$ to the trace's Allan deviation to predict the noise of the image's N = 500-skip average; gain = training mean.	36.1/29.2/31.3
physics (A), default *(Allan curve + numbers)*	trace_physics + GPs that learn the gain and a correction for what the trace physics misses (outlier packets down-weighted). λ by a **SAAS GP**.	9.1/7.4/0.9
physics_joint *(as A + λ per row)*	As A, but λ = dark current + CIC fitted jointly from the image's row-by-row λ (GP with the SAAS prior).	9.1/4.6/0.9
sets (E) *(each trace set)*	As A, but looks at every repeated trace set and learns which to trust: robust to damaged sets.	5.7/10.9/0.9
Networks that learn their own features (the 14 curves)		
cnn *(14 curves)*	1D convolutional network on the curves (Allan deviation, spectra, waveform shapes).	7.6/13.1/2.0
pretrained (D) *(14 curves)*	The CNN, first pre-trained on unlabelled traces (reference traces need no image), then fine-tuned.	6.8/17.2/2.6
deep_kernel (C) *(14 curves)*	Gaussian process on the pre-trained network's summary of the curves: proper error bars.	8.5/20.7/0.9
Reference only		
physics_hier (A+B) *(as A + amplifier)*	A told the module and amplifier (hierarchical GPs): measures what amplifier labels would add.	9.1/10.6/0.9

What is a SAAS GP?

Gaussian process (GP). Predicts a value at new settings as a weighted average of the training examples, weighted by how similar their inputs are, and returns an error bar. Similarity is the kernel $k(x, x') = s^2 \exp(-\tfrac{1}{2} \Sigma_i (x_i - x'_i)^2 / \ell_i^2)$, with one **length scale** ℓ_i per input: a short ℓ_i means the input matters, a long one that it is ignored.

The problem. Our GPs see 60–320 inputs but only 40–400 examples. Fitting every ℓ_i freely gives more knobs than data: the GP latches onto chance patterns.

SAAS (sparse axis-aligned subspace; Eriksson & Jankowiak, UAI 2021, arXiv:2103.00349) puts a **sparsity prior** on the inverse squared length scales: $\rho_i = 1/\ell_i^2 \sim \text{HalfCauchy}(\tau)$, with a global scale $\tau \sim \text{HalfCauchy}(0.1)$. Most ρ_i are pulled to ~0 (input switched off); the heavy tail lets the few inputs the data really need stay on. Instead of one best fit, the hyperparameters are **sampled** (NUTS, Markov chain Monte Carlo) and the predictions averaged, so the error bar includes the uncertainty about which inputs matter. A by-product: the inputs it keeps are a feature-relevance ranking.

In this project (src/ml/bogp.py, BoTorch *SaasFullyBayesianSingleTaskGP*): **physics** fits λ with a SAAS GP (short chains: 96 warm-up / 48 samples, ~75 s per run at 100 packets); **physics_joint** uses the same prior fitted by maximum a posteriori (no sampling, no extra time). Synthetic λ error at 13 / 41 / 100 packets: physics 19.1 → 15.1, 10.9 → 7.4, 4.9 → 3.9 %; physics_joint 17.3 → 16.9, 5.2 → 4.6, 4.6 → 4.4 %. σe and gain unchanged. Not adopted: SAAS or the dimension-scaled prior for the plain *gp* (no gain there).